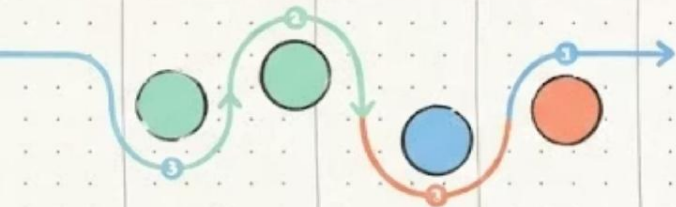


Gestão do Ciclo de Vida da Aplicação

Processos de Desenvolvimento de Software

Professor Romário da Silva Borges



Uma equipe recebeu a tarefa de desenvolver um sistema, mas o cliente ainda não sabe exatamente tudo o que precisa. Como organizar esse trabalho?

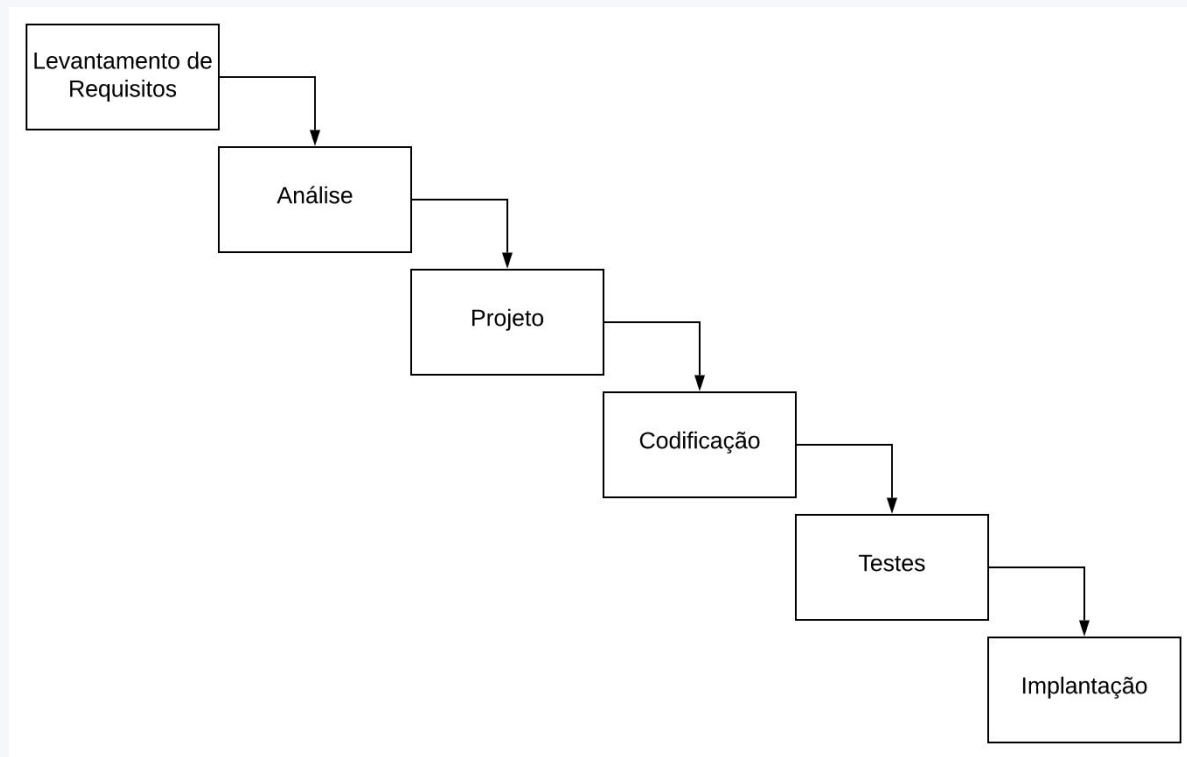
Processo de Desenvolvimento de Software

The background features a series of flowing, wavy lines in shades of blue and purple, creating a sense of movement and depth. Interspersed within these waves are faint, semi-transparent snippets of code in various programming languages, including JavaScript, Python, and C++, which are slightly blurred to maintain focus on the title.

Engenharia Tradicional

- Civil, mecânica, elétrica, aviação, automobilística, etc
- Projeto com duas características:
 - Planejamento detalhado (*big upfront design*)
 - Sequencial
- Isto é, Waterfall, há milhares de anos

Modelo em Cascata



O modelo em cascata (*Waterfall*) foi um dos primeiros modelos formais de processo de desenvolvimento. Proposto por Royce em 1970, organiza o desenvolvimento como uma **sequência linear de fases**, onde cada fase deve ser concluída antes que a próxima comece.

No entanto: Waterfall não funcionou com software!

Software é diferente

- Engenharia de Software $\neq$ Engenharia Tradicional
- Software $\neq$ (carro, ponte, casa, avião, celular, etc)
- Software $\neq$ (produtos físicos)
- Software é abstrato e "adaptável"

Dificuldade 1: Requisitos

- Clientes não sabem o que querem (em um software)
 - Funcionalidades são "infinitas" (difícil prever)
 - Mundo muda!
- Não dá mais para ficar 1 ano levantando requisitos, 1 ano projetando, 1 ano implementando, etc
- Quando o software ficar pronto,
ele estará obsoleto!

Dificuldade 2: Documentações Detalhadas

- Verbosas e pouco úteis
- Na prática, desconsideradas durante implementação
- *Plan-and-document* não funcionou com software



Modelo em Cascata

Características

- Fases bem definidas e sequenciais
- Documentação extensa em cada etapa
- Pouca tolerância a mudanças
- Fácil de gerenciar e compreender

Vantagens

- Estrutura clara e disciplinada
- Planejamento detalhado antecipado
- Adequado para requisitos estáveis
- Facilita estimativas de custo e prazo

Desvantagens

- Mudanças são caras e difíceis
- O cliente só vê o produto no final
- Erros de requisitos têm alto custo
- Não reflete a realidade de projetos complexos

Desenvolvimento Ágil

The background features a series of fluid, overlapping waves in shades of light blue and lavender. These waves create a sense of motion and flow. Interspersed within these waves are faint, semi-transparent snippets of code in various programming languages, including JavaScript, Python, and PHP. The code is not the primary focus but adds a technical, digital feel to the overall aesthetic.

Contexto de Surgimento dos Métodos Ágeis

No final dos anos 1990, a indústria de software vivia uma **crise de projetos**: altas taxas de falha, estouros de prazo e custo, e entrega de sistemas que não atendiam às reais necessidades dos usuários. Esse cenário motivou profissionais a repensarem as abordagens tradicionais.

Anos 1970–1980

Dominância do modelo em cascata. Foco em documentação pesada e processos rígidos inspirados na engenharia civil e manufatura.

Fevereiro de 2001

Dezessete desenvolvedores se reúnem em Snowbird, Utah (EUA), e redigem o **Manifesto para Desenvolvimento Ágil de Software**, unificando os princípios das novas abordagens.

1

2

3

4

Anos 1990

Relatórios do Standish Group (Chaos Report) revelam que mais de 50% dos projetos falham ou são entregues fora do prazo e orçamento. Surgem metodologias leves como XP, Scrum e Crystal.

Anos 2000 em diante

Adoção crescente dos métodos ágeis na indústria. Scrum e XP tornam-se padrão em empresas de tecnologia ao redor do mundo.

Manifesto Ágil (2001)



Manifesto Ágil

 agilemanifesto.org



Princípios por trás do Manifesto Ágil

Nossa maior prioridade é satisfazer o cliente através da entrega contínua e adiantada de software com valor agregado.

12 Princípios do Manifesto Ágil

Satisfação do cliente

Entregar software de valor continuamente, em intervalos curtos, é a maior prioridade.

Bem-vinda às mudanças

Mudanças de requisitos são aceitas mesmo tardiamente — elas geram vantagem competitiva para o cliente.

Entregas frequentes

Software funcionando deve ser entregue com frequência, de semanas a poucos meses.

Colaboração em todo o projeto

Desenvolvedores e pessoas de negócio trabalham juntos diariamente durante o projeto.

Indivíduos motivados

Projetos são construídos em torno de pessoas motivadas, com o ambiente e suporte de que precisam.

Melhoria contínua e reflexão

A equipe reflete regularmente sobre como se tornar mais eficaz e ajusta seu comportamento.

12 Princípios do Manifesto Ágil

Continuando os princípios que orientam o desenvolvimento ágil de software:

Comunicação eficaz

A forma mais eficiente e eficaz de transmitir informação para e dentro de uma equipe de desenvolvimento é a conversa face a face.

Software funcionando

Software funcionando é a principal medida de progresso. Priorizamos entregas que podem ser usadas e testadas.

Ritmo sustentável

Processos ágeis promovem um ritmo de desenvolvimento sustentável. Patrocinadores, desenvolvedores e usuários devem ser capazes de manter um ritmo constante indefinidamente.

Excelência técnica

A atenção contínua à excelência técnica e ao bom design aumenta a agilidade e a qualidade do produto.

Simplicidade

A arte de maximizar a quantidade de trabalho não feito é essencial. Menos é mais, focando no valor.

Equipes auto-organizadas

As melhores arquiteturas, requisitos e designs emergem de equipes auto-organizadas e colaborativas.

Desenvolvimento iterativo

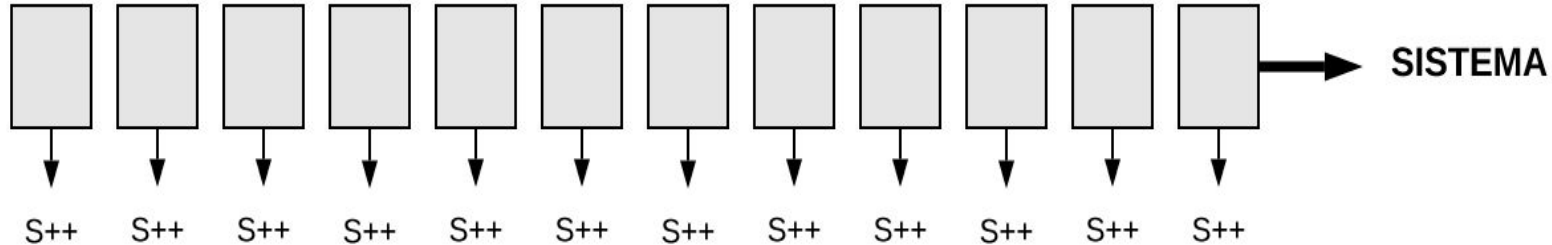
- Suponha um sistema imenso, complexo etc
- Qual o menor "incremento de sistema" eu consigo implementar em 15 dias e validar com o usuário?
- Validar é muito importante!
- Cliente não sabe o que quer!

Ideia central: desenvolvimento iterativo

Waterfall



Ágil



Reforçando: **ágil = iterativo**

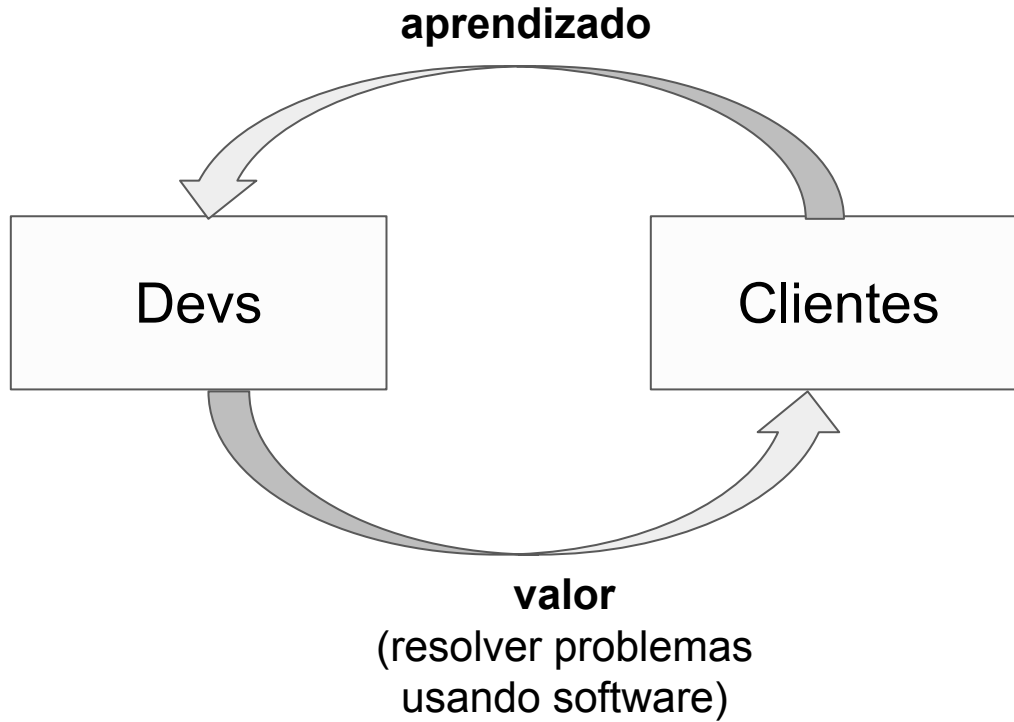
Outros pontos importantes (1)

- Menor ênfase em documentação
- Menor ênfase em *big upfront design*
- Envolvimento constante do cliente

Outros pontos importantes (2)

- Novas práticas de programação
 - Testes, refactoring, integração contínua, etc

Agilidade = aprendizado + geração de
valor contínuos



Métodos Ágeis: XP



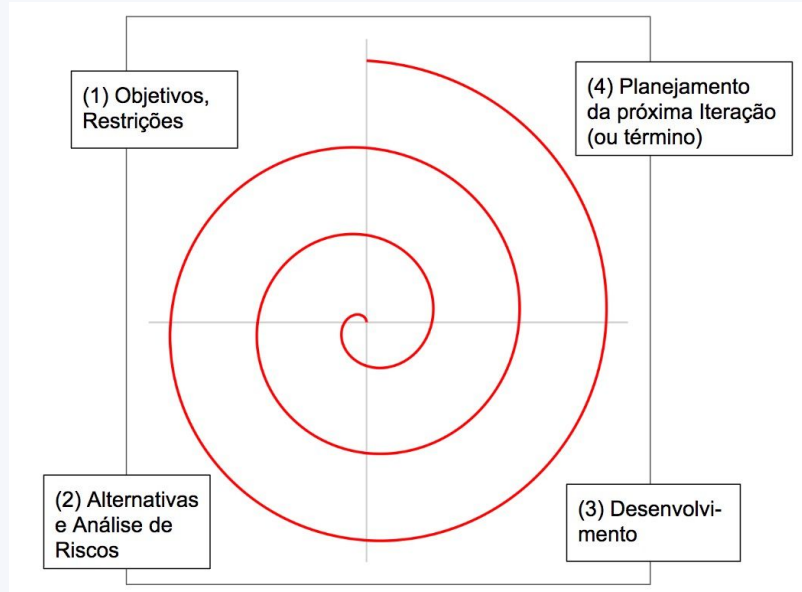
Extreme Programming (XP)

Focado em práticas técnicas para entregar software de alta qualidade rapidamente e se adaptar às mudanças dos clientes. As principais práticas incluem programação em par, desenvolvimento orientado a testes e integração contínua.

Outros métodos

A transição entre Waterfall — dominante nas décadas de 70 e 80 — e métodos ágeis — que começaram a surgir na década de 90, mas que só ganharam popularidade no final dos anos 2000 — foi gradativa. Como em métodos ágeis, esses métodos surgidos nesse período de transição possuem o conceito de iterações. Ou seja, eles não são estritamente sequenciais, como em Waterfall. Porém, **as iterações têm duração maior do que aquela usual em desenvolvimento ágil**. Em vez de poucas semanas, elas **duram alguns meses**. Por outro lado, eles preservam características relevantes de Waterfall, como **ênfase em documentação** e em uma **fase inicial de levantamento de requisitos e depois de design**.

Outros métodos - Modelo em Espiral



É focado em **gestão de riscos**. Nesse modelo, um sistema é desenvolvido na forma de uma espiral de iterações. Cada iteração, ou volta completa na espiral, inclui quatro etapas.

Cada iteração, versões mais completas de um sistema, começando da versão gerada no centro da espiral. Porém, cada iteração, somando-se as quatro fases, pode levar de 6 a 24 meses.

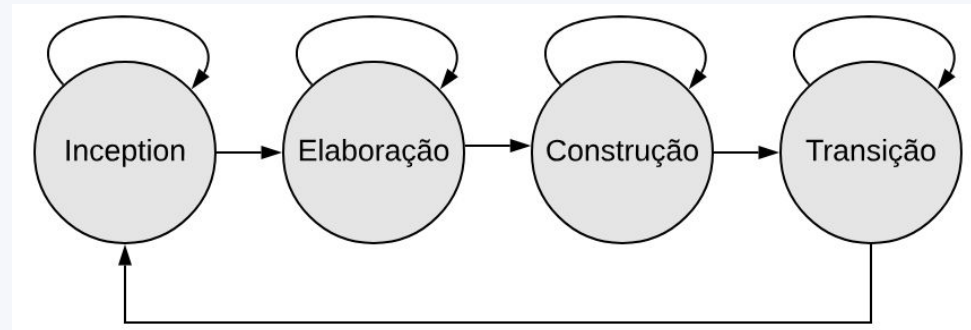
Outros métodos - Modelo em Espiral

- **Definição de objetivos e restrições**, tais como custos, cronogramas, etc.
- **Avaliação de alternativas e análise de riscos**. Por exemplo, pode-se chegar à conclusão de que é mais interessante comprar um sistema pronto, do que desenvolver o sistema internamente.
- **Desenvolvimento e testes**, por exemplo, usando Waterfall. Ao final dessa etapa, deve-se gerar um protótipo que possa ser demonstrado aos usuários do sistema.
- **Planejamento da próxima iteração** ou então tomar a decisão de parar, pois o que já foi construído é suficiente para atender às necessidades da organização.

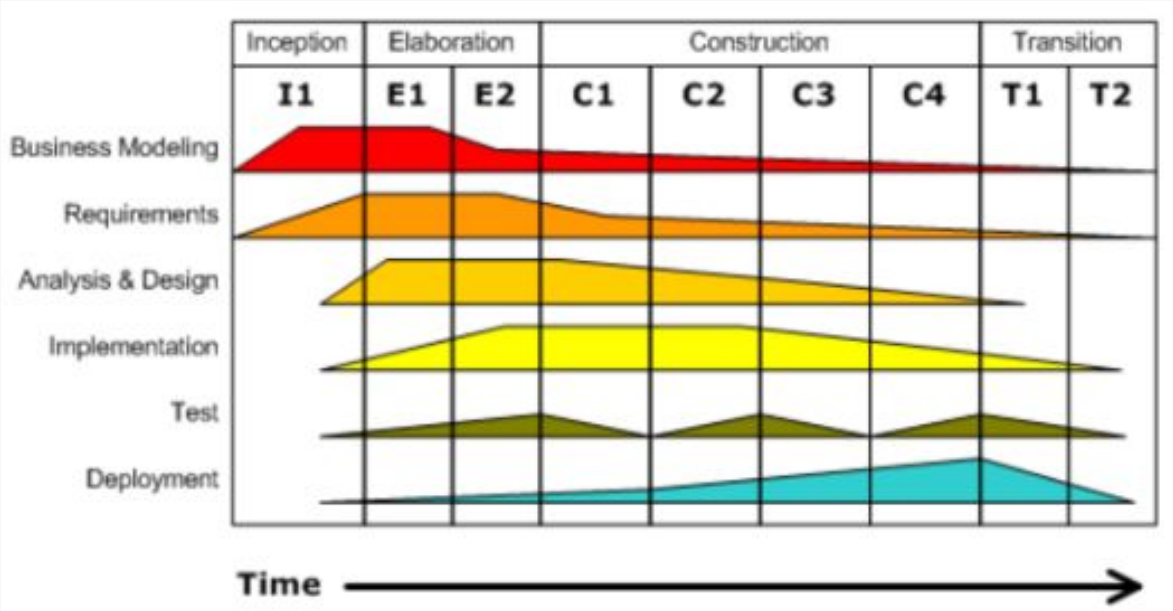
Outros métodos - Processo Unificado (UP)

Define-se um conjunto de disciplinas de engenharia que incluem por exemplo: modelagem de negócios, definição de requisitos, análise e design, implementação, testes e implantação. Essas disciplinas — ou fluxos de trabalho — podem ocorrer em qualquer fase. **O UP antecipa os riscos arquiteturais logo no início.**

Assim como no Modelo Espiral, pode-se repetir várias vezes o processo; ou seja, o desenvolvimento é incremental, com novas funcionalidades sendo entregues a cada ciclo.

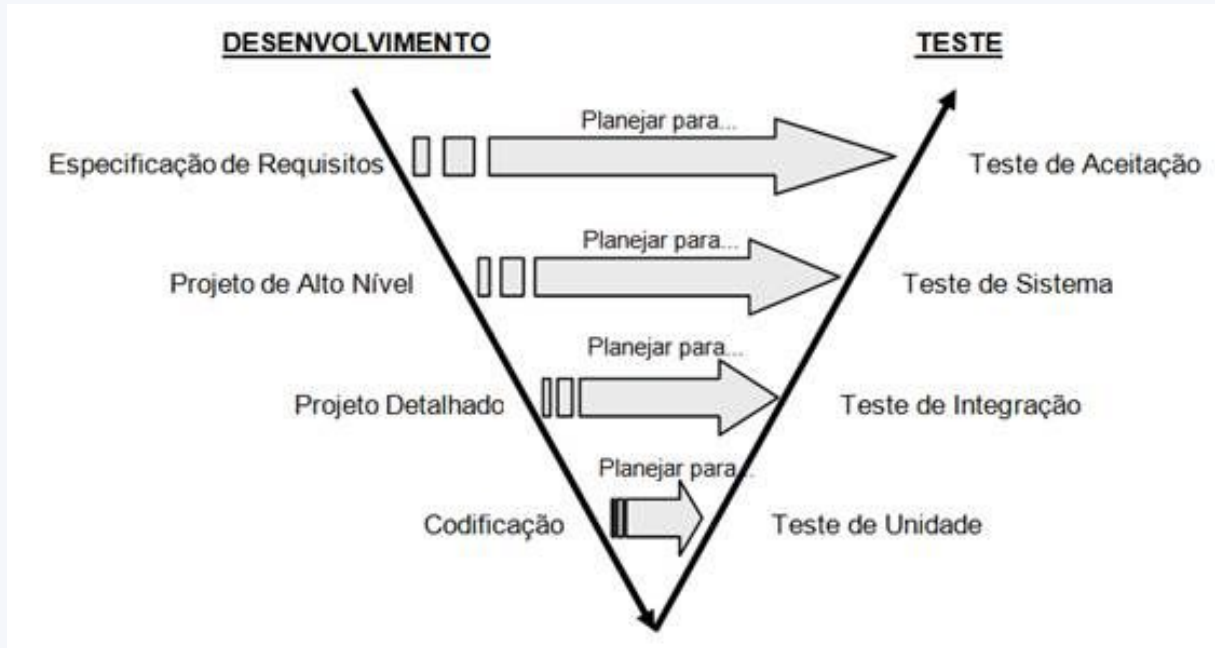


Outros métodos - Processo Unificado (UP)



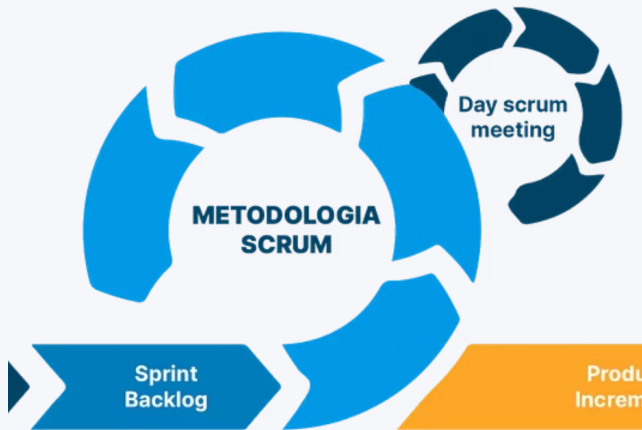
Outros métodos - Modelo V

Funciona de forma linear. Você precisa terminar completamente uma fase de especificação antes de avançar



Característica	Modelo V	Modelo Espiral	UP (Unified Process)	XP (Extreme Programming)
Rigidez	Alta (Linear/Preditivo)	Baixa (Evolutivo)	Média/Baixa (Iterativo)	Quase Nula (Adaptativo)
Foco Principal	Testes e Qualidade nominal	Mitigação de Riscos	Arquitetura e Casos de Uso	Práticas de Engenharia e Código
Ciclo de Feedback	Apenas no final do projeto	A cada ciclo da espiral	Ao fim de cada incremento	Contínuo (Diário ou Semanal)
Mudanças de Escopo	Difíceis e muito caras	Naturais a cada ciclo	Aceitas entre iterações	Bem-vindas a qualquer momento

Métodos Ágeis: Scrum



Framework iterativo para gerenciar projetos complexos. Foca em equipes auto-organizadas, papéis definidos, sprints com tempo fixo e ciclos de feedback contínuos para entregar software funcionando de forma incremental.

Métodos Ágeis: Kanban



Um método visual para gerenciar o trabalho, que permite às equipes ver o progresso, identificar gargalos e limitar o trabalho em andamento (WIP). Promove a melhoria contínua e a flexibilidade na entrega.

Professor Marco Tulio Valente

Slides disponibilizados pelo Prof. Marco Tulio Valente. Esta apresentação contém adaptações da apresentação original que está disponível na seguinte página: <https://engsoftmoderna.info>.

BEZERRA, E.

Princípios de Análise e Projeto de Sistemas com UML, 2a ed., Editora Campus/Elsevier: Rio de Janeiro, 2007

PAULA, Filho W. P.

Engenharia de Software: Fundamentos, Métodos e Padrões, 3a ed. Ed. LTC: Rio de Janeiro, 2009.

HORSTMANN, C.

Padrões e Projeto Orientados a Objetos, 2 Edição, Editora Bookman: Porto Alegre, 2007

SOMMERVILLE, I.

Engenharia de Software. 8a ed., Ed. Pearson: São Paulo, 2008.

PRESSMAN, R.

Software Engineering: A Practitioner's Approach. 5a ed. McGraw Hill : São Paulo, 2000

